

Lab 3 report

实验目的与内容

为上次设计的 CPU 增加了分支跳转功能和访存功能。

任务 1. 访存控制单元设计

```
wire [4:0] offset = addr [1:0] << 3;
```

首先，因为一次会读 4 个字节，先计算出 addr 对应的偏移量。

```
always @(*) begin
    case(dmem_access)
        `SD_LDB:begin
            rd_out = {{24{rd_in[offset + 7]}}, rd_in[offset +: 8]};
            wd_out = wd_in;
        end
        `SD_LDH:begin
            case(offset[4:3])
                2'b00:rd_out = {{16{rd_in[15]}}, rd_in[15:0]};
                2'b10:rd_out = {{16{rd_in[31]}}, rd_in[31:16]};
                default:rd_out = rd_in;
            endcase
            wd_out = wd_in;
        end
        `SD_LDW:begin
            rd_out = rd_in;
            wd_out = wd_in;
        end
    end
end
```

```

`SD_LDBU:begin
    wd_out = wd_in;
    rd_out = {{24{1'b0}}, rd_in[offset+:8]};
end
`SD_LDHU:begin
    case(offset[4:3])
        2'b00:rd_out = {{16{1'b0}}, rd_in[15:0]};
        2'b10:rd_out = {{16{1'b0}}, rd_in[31:16]};
        default:rd_out = rd_in;
    endcase
    wd_out = wd_in;
end

```

Load 指令：LD.B(U)时读偏移量起的一个字节，LD.H(U)时对偏移量分类讨论，读前半字/后半字，LD.W 时直接返回整个字即可。

```

`SD_STB:begin
    rd_out = rd_in;
    case(offset[4:3])
        2'b00:wd_out = {rd_in[31:8], wd_in[7:0]};
        2'b01:wd_out = {rd_in[31:16], wd_in[7:0], rd_in[7:0]};
        2'b10:wd_out = {rd_in[31:24], wd_in[7:0], rd_in[15:0]};
        2'b11:wd_out = {wd_in[7:0], rd_in[23:0]};
        default:wd_out = wd_in;
    endcase
end
`SD_STH:begin
    rd_out = rd_in;
    case(offset[4:3])
        2'b00:wd_out = {rd_in[31:16], wd_in[15:0]};
        2'b10:wd_out = {wd_in[15:0], rd_in[15:0]};
        default:wd_out = wd_in;
    endcase
end
`SD_STW:begin
    rd_out = rd_in;
    wd_out = wd_in;
end

```

Load 指令：ST.B 和 ST.H 对偏移量分类讨论，将写的内容与读到的值合成，ST.W 时直接返回要写的整个字即可。

任务 2. 搭建 CPU

BRANCH 模块：对 bt_type 分类讨论即可。


```

stone_heart0027@DereksPC:~/COS_lab/Lab4/COD25-Simulation-Framework$ ./build/sim
Simulation started...
Difftest level: FULL.
Dumping waveform to waveform/waveform.vcd.
Hit good trap.
Simulation finished.
Time: 4456
Instruction count: 2227
PC: 0x1C0023A8
Instruction: 0x80000000
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x1C002190 [ 2] = 0x000000BF [ 3] = 0x1BFF8E28
[ 4] = 0x00000000 [ 5] = 0x00000000 [ 6] = 0x1C002110 [ 7] = 0x1C002190
[ 8] = 0x1C824745 [ 9] = 0x00000000 [10] = 0x1C017504 [11] = 0x1C002130
[12] = 0x1C811381 [13] = 0x1C0020B0 [14] = 0x00000000 [15] = 0x00000043
[16] = 0x1C002130 [17] = 0x1C015B84 [18] = 0x1BFE4894 [19] = 0x000000BF
[20] = 0x1C8258D3 [21] = 0x1C002090 [22] = 0x1C001FC8 [23] = 0x00000000
[24] = 0x1C0021B0 [25] = 0x1BFFB0C4 [26] = 0x1C002170 [27] = 0x0000006F
[28] = 0x1C0021B0 [29] = 0x1C002028 [30] = 0x1BFED374 [31] = 0x1C0020F0

```

上板：

```

User: R?????????AU-----

User: RR 0 32;
00000000 C036D7CC 0000082F 00000000
00000407 A81FB808 C036D4A4 00000000
A9D9B000 00000001 00000001 00000000
BFB4A4BC 00000000 00000000 35176000
00000000 BFB4A7F4 6E659000 048DA95B
00000643 BB0D3498 718544B0 5C703000
BB0D37B8 A81FB4C8 00000001 5C703000
A6503C01 917E1000 00000000 A9D9B000

```

```
FPGAOL UART xterm.js 1.1

USTC COD Project
User: R?????????AU-----

User: RR 0 32;
00000000 1C000424 0000006C 0000006C
00000000 1C000424 0000006C 1C000444
0000006C 0000006C 1C80F723 1C809826
1C8101B3 0000006C 00000000 0000006C
0000006C 0000006C 1C000444 0000006C
00000000 0000006C 6EB705E2 0000006C
1C809F56 1C804CD7 1BFEB73C 0000006C
00000000 1C8100BD 0000006C F3AF74E4
```

R20 / T8 0x0	R21 0x6c	R22 / S9 / FP 0x6eb705e2	R23 / S0 0x6c
R24 / S1 0x1c809f56	R25 / S2 0x1c804cd7	R26 / S3 0x1bfeb73c	R27 / S4 0x6c
R28 / S5 0x0	R29 / S6 0x1c8100bd	R30 / S7 0x6c	R31 / S8 0xf3af74e4

如图，与 LARS 结果一致。

任务 3. 斐波那契额数列

```

Hit good trap.
Simulation finished.
Time:                                660
Instruction count:                    329
PC:                                  0x1C000050
Instruction:                          0x80000000
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x00000000 [ 2] = 0x0000001E [ 3] = 0x00000000
[ 4] = 0x000CB228 [ 5] = 0x00000000 [ 6] = 0x0007D8B5 [ 7] = 0x00000000
[ 8] = 0x000CB228 [ 9] = 0x0000001E [10] = 0xFFFFFFFF [11] = 0xFFFF8274A
[12] = 0x00000000 [13] = 0x00000000 [14] = 0x00000000 [15] = 0x00000000
[16] = 0x00000000 [17] = 0x00000000 [18] = 0x00000000 [19] = 0x00000000
[20] = 0x00000000 [21] = 0x00000000 [22] = 0x00000000 [23] = 0x00000000
[24] = 0x00000000 [25] = 0x00000000 [26] = 0x00000000 [27] = 0x00000000
[28] = 0x00000000 [29] = 0x00000000 [30] = 0x00000000 [31] = 0x00000000

```

```

USTC COD Project
User: R?????????AU-----

User: RR 0 32;
00000000 00000000 0000001E 00000000
000CB228 00000000 0007D8B5 00000000
000CB228 0000001E FFFFFFFF FFF8274A
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000

```

R0 / ZERO 0x0	R1 / RA 0x0	R2 / TP 0x1e	R3 / SP 0x0
R4 / A0 0xcb228	R5 / A1 0x0	R6 / A2 0x7d8b5	R7 / A3 0x0
R8 / A4 0xcb228	R9 / A5 0x1e	R10 / A6 0xffffffff	R11 / A7 0xffff8274a

任务 4. 思考与总结

1.如图，SLU 增加一个 mask 输出，接入存储器的 mask 信号。

output	reg	[3 : 0]	mask
--------	-----	---------	------

对于读操作，若使用掩码，依然需要处理符号拓展以及低位的 0 的问题，所以直接采取原设计即可。

```
`SD_STB:begin
    rd_out = rd_in;
    wd_out = wd_in << addr[1 : 0];
    mask = 4'b0000;
    mask[addr [1 : 0]] = 1'b1;
end
`SD_STH:begin
    rd_out = rd_in;
    wd_out = wd_in << addr[1 : 0];
    mask = 4'b0000;
    mask[addr [1 : 0]] = 1'b1;
    mask[addr [1 : 0] ^ 1] = 1'b1;
end
`SD_STW:begin
    rd_out = rd_in;
    wd_out = wd_in;
    mask = 4'b1111;
end
```

对于写操作，考虑利用掩码，改动如图。wd_out 为 wd_in 左移 addr[1:0]位，方便合成所需写入数据。然后根据 addr[1:0]的值，分 ST.B 和 ST.H 两种情况确定 mask 的值即可。

2.可能的关键路径：

①Load 指令路径：PC -> IM -> Decode -> 寄存器堆读 -> ALU -> DM 读 -> SLU -> 写回选择 MUX -> 寄存器堆写

②条件分支指令路径：PC -> IM-> Decode -> 寄存器堆读 -> BRANCH/ALU 计算目标地址 -> NPCMUX -> PC

总结

这次实验为上次设计的 CPU 增加了分支跳转功能和访存功能，受益匪浅。